



RUNNRS

Run Club Management Application

Web App Documentation



Contents

1. Application Description (Mo)	2
app deliverables	2
Requirement specifications	2
2. Conceptual Framework (Mo)	3
3. Classes, Methods & Features (Byron)	4
A. Classes	4
B. Controllers	4
C. Data Access Layer	5
D. Methods	6
E. Advanced Features	8
4. Constraints & Strategies (Byron & Mo)	8

1. Application Description (Mo)

Description of the application that you are building, deliverables and the requirement specifications.

Our idea for Runrs is an app that helps people connect with their community and get healthier by joining their local running clubs. Runrs does this by letting users create and join clubs in the app and letting them have events and announcements.

Each club will have details like location, running difficulty, casual or professional and public or private. If a club is private and a user wants to join, they must first get approved by the club owner. The app will also have a search function where users can search for clubs in their towns and cities or just brows.

app deliverables

- Built with ASP.NET Core MVC
- CRUD operations
- Search functionality
- Login and Registration
- Login hashing
- Map functionality
- Using Repository Pattern
- Using Unit of Work
- Using Bootstrap

Requirement specifications

Authentication

- Users can make a new account
- Users can login and logout
- Users' password is hashed

Clubs

- Users can create, update, and delete clubs
- Clubs can have details like name description, location, and difficulty

Membership

- Users join and leave clubs
- Membership is tracked (Pending, Approved, Rejected)

Search

- Users can search for names or locations of clubs

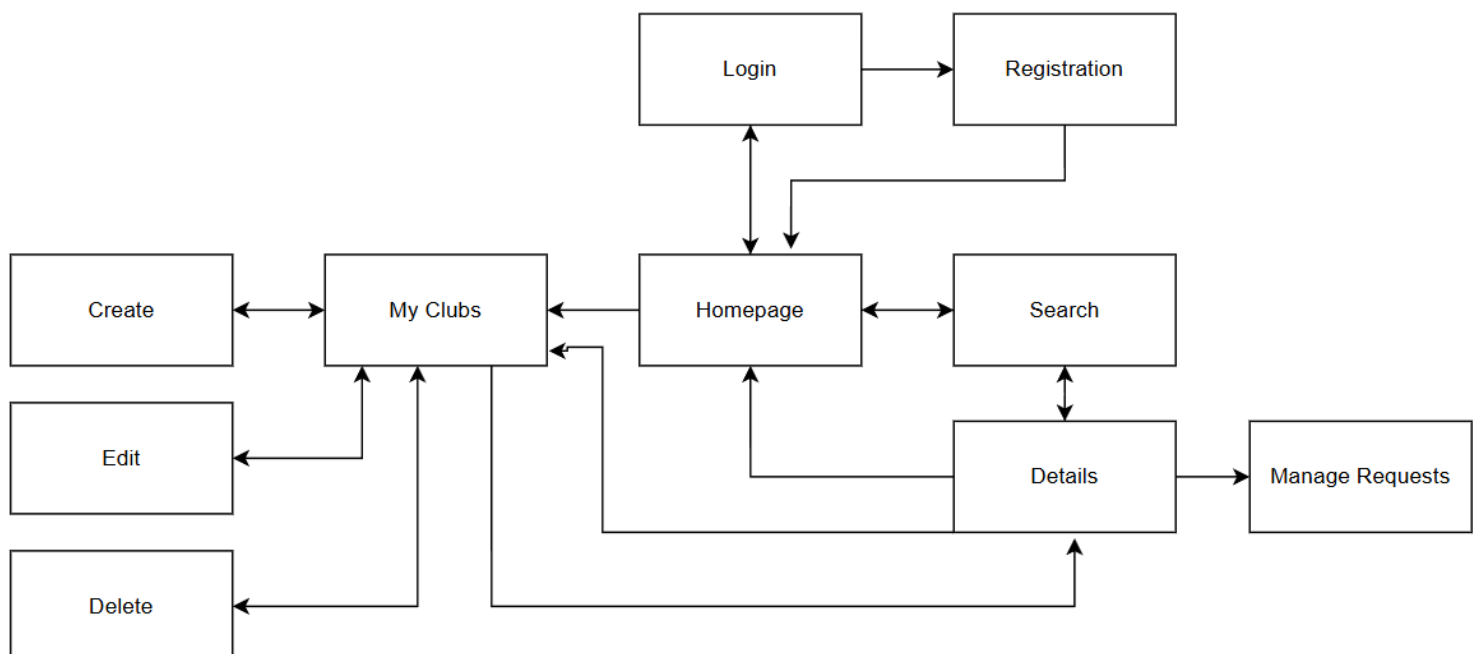
2. Conceptual Framework (Mo)

Framork were using

Runrs uses the MVC framework to structure the app code. MVC (Model View Controller) is a design pattern that separates code into three parts: Model, View, and Controller.

The Model is in charge of data and business logic of the app and handling requests from the View and Controller. The view is of charge of the displaying the app pages, it does this by getting data from the Model and user inputs from the Controller. The Controller acts as an in-between for the Model and View by sending user data to the model and updating the View.

Runrs is broken up this way so that it is easier to focus on specific parts of the app without. This way of structuring the app follows a principle of MVC called separation of concerns(SoC). The idea behind SoC is different parts of the app should have different responsibilities and that they should not overlap. This way of coding also makes it easier to test and maintain the app. Since the code is all separate if there is a bug it's easier to find. Another benefit is that this way of structuring code makes it easier to scale up the app if we need to.



3. Classes, Methods & Features (Byron)

A. Classes

User.cs: The User model class consists of the user properties including ***Username***, ***FirstName***, ***LastName***, ***Email***, ***PasswordHash*** and ***DateOfBirth***. Each user account that is created will provide these values on register. The User class also contains the ***ICollection<Membership>*** and ***ICollection<Club>*** properties which are used for the one-to-many relationships with the Membership.cs and Club.cs classes.

Club.cs: The Club model class represents any run club that is created in the Runrs application. The properties in Club.cs include ***ClubName***, ***ClubDescription***, ***ClubLocation***, ***IsPrivate***, ***CreatedAt***, ***OwnerId*** and ***ImageUrl***. We have also implemented 3 enums named ***DifficultyLevel***, ***DistanceRange*** and ***ClubType*** that are used to store various constants to categorise the run clubs.

Membership.cs: The Membership model class contains the UserId and ClubId as foreign keys creating a link to their respective models. The class also contains the ***Role*** and ***Status*** properties to define the user's role in a run club (Admin or Member) and track the users membership status for a club (pending, approved or rejected). This model class leverages the Entity Framework, facilitating the many-to-many relationships between Club.cs and User.cs.

ApplicationDbContext.cs: The ApplicationDbContext class enables us to use the SQL database within our project by initializing three "DbSet" properties named Users, Clubs and Memberships. These three properties make up our database tables and are used to store the properties located in the User.cs, Club.cs and Membership.cs model classes. Also located in the DbContext is our seed data for the User and Club tables, enabling us to pass working data into our web application to refer to and test.

B. Controllers

BaseController.cs: The BaseController has been implemented into the Runrs application for authentication purposes. It ensures that any action made by all other controllers is checked to ensure an authenticated user exists in a verified HTTP session.

HomeController.cs: Home Controller handles the logic located on the home page of our web application. It defines the Index() action that is used to display the static contents of the home page. The homepage doesn't contain any functionalities or in-depth logic as it is intended to be an "About us" type page explaining what our app is and what it is used for. The Error() action is also located in the HomeController and is used for handling any application errors through the ErrorViewModel.

LoginController.cs: The LoginController handles the authentication and registration functionalities in our application. Like the other controllers, the LoginController contains an Index() action to display the Login View. Along with Index is the Registration() post action

that is used to create a new user who inputs their details in the registration screen. Upon submitting new user details in the registration screen, the system will automatically log in the new user and store the user's data in the User table of our database. We also have the Login() post action that logs a valid user into the application if the username and password are valid.

ClubController.cs: ClubController handles all club related actions/methods and is used heavily throughout our application. The controller uses *UnitOfWork* to access all three repositories defined in our repository pattern, allowing for full **CRUD (Create, Retrieve, Update, Delete)** operations related to clubs. We have also implemented validations into the ClubController, ensuring only club owners can perform significant actions such as deleting or editing run clubs. Along with CRUD actions, the ClubController also contains the **Details()** action which is used to load club details from either the "Search" or "My Clubs" pages. Furthermore, are the **Join()** and **Leave()** actions; two actions used for creating or removing memberships to run clubs.

SearchController.cs: The SearchController handles all operations for the "Search" page, allowing for users to search for and view run clubs created on the Runrs application. The controller also leverages the UnitOfWork pattern for access to the three repositories. The **Index()** action retrieves all the created clubs in the application by default. The search query input on the search page can be used to filter run clubs by either club name, club location or keywords found in the club description. The SearchController also uses **includeProperties: "Owner"** to load the owner's username in the club cards.

C. Data Access Layer

IRepository<T>: The IRepository interface implementation defines the "core" CRUD operations for all entity types in the application. It leverages the **<T> where T: class** parameter to ensure the interface can work with User, Club and Membership reference types.

IClubRepository: This interface inherits the generic CRUD operations **GetAll(), Get(), Add(), Remove()** and **RemoveRange()** from **IRepository<Club>**. It also contains the **Update(Club obj)** and **Save()** methods. This ensures that our controllers are provided a clean interface for club operations without the need for duplicating code.

IUserRepository: Comparable to IClubRepository, the IUserRepository interface "extends" the IRepository<User> as well as adding user functionalities like **GetByUsername(string username)**. This method is used during the authentication phase of logging in and allows the LoginController to validate the user's login credentials (Username & PasswordHash).

IMembershipRepository: The IMembershipRepository interface provides query methods for managing the relationships between Users and Clubs (many-to-many). Methods include **GetMembershipByUserId(), GetMembershipByClubId()** and **IsMember()**. Each of these methods returns a Boolean or collection of membership records in order to verify or retrieve

membership data. The interface also contains **Update (Membership obj)** and **Save()**, similar to the IClub and IUser repositories.

IUnitOfWork: We have implemented the UnitOfWork design pattern, allowing access to our three repositories: **Club**, **User** and **Membership**. The interface also contains the Save() method for executing and saving any pending changes across the three repositories to our Runrs database (transaction). This pattern ensures data consistency across our application and further separates our business logic from the data access layer (decoupling).

D. Methods

Method	Description
BaseController	
OnActionExecuting()	Used to checked for authenticated users
HomeController	
Index()	Used to display our static homepage
Error()	Used to handle any errors via the ErrorViewModel
LoginController	
Index()	Displays the Runrs Login Page
Login() POST	A post method that is used to validate a user's username and password
Logout()	Clears an active session and sends user back to login screen
Registration()	Used to display the Runrs registration page
Registration(User) POST	An action method used for creating a new user via the registration screen.
ClubController	
Index()	Displays the MyClub page, which consists of two tables that display the clubs the user either owns or has joined.
Create() POST	Displays the form for creating a new club.
Create()	Post method used when creating a new club and sets the user as the owner.
Edit()	Displays the edit form for editing existing clubs that the user owns.
Edit() POST	Post method that updates a club when an owner makes an edit.
Delete()	Displays the club form with all input fields and dropdowns disabled.
DeletePost() POST	Post method for deleting a club. Like the edit method, this method can only be actioned if the user is the club owner.

Details()	Used to display the details of a club, the users membership status (pending, approved or rejected). It is also used to display the list number of members in a club.
Join()	A post method actioned when user clicks join on a club. The method will either set membership status as approved or pending, based on if the club is public (immediate approved) or private (pending approval).
Leave()	A post method that removes a user's membership to a club when they leave.
ManageRequests()	A method exclusive to club owners that displays a club owner pending join requests.
ApproveRequest()	An owner exclusive post method for approving any pending join requests to a club.
RejectRequest()	An owner exclusive post method for rejecting any pending join requests to a club.
SearchController	
Index()	Displays all clubs in the search page which can be filtered via a search query (name, location, or description).
Repository<T>	
Add()	Used to add any entities to our DbSet
GetAll()	Used to return all entities
Get()	Used to return an entity matching filter
Remove()	Used to remove an entity from the DbSet
RemoveRange()	Removes a range of entities from the DbSet
ClubRepository	
Save()	Used to call the SaveChanges() method in the ApplicationDbContext ensuring changes are saved to the Runrs database.
Update()	Used to update an existing club entity in the DbSet.
UserRepository	
Save()	Used to call the SaveChanges() method in the ApplicationDbContext ensuring changes are saved to the Runrs database.
Update()	Used to update an existing user entity in the DbSet.
GetByUsername()	Used to return the user matching the username.
MembershipRepository	
Save()	Used to call the SaveChanges() method in the ApplicationDbContext ensuring changes are saved to the Runrs database.

Update()	Used to update an existing membership entity in the DbSet.
GetMembershsipByUserId()	Used to return all memberships for a user.
GetMembershsipByClubId()	Used to return all memberships for a club.
IsMember()	This method returns true if a user has a membership for a club.
UnitOfWork	
Save()	Used to call the SaveChanges() method in the ApplicationDbContext ensuring changes are saved to the Runrs database.

E. Advanced Features

1) Search Query Feature (Byron)

A search query feature has been added in the Search page of the Runrs application. Any clubs can be discovered or found using the input box. Clubs can be filtered when a keyword is searched in the input box. Clubs can be filtered based on if a keyword is in the club description, the club location or the club's name. The SearchController contains the Index() action method that which handles the search feature by accepting a string parameter (keyword) from the input box. It implements the GetAll() method if the input box is searched while empty or searches all clubs that feature the string query. With the assistance of the UoW pattern, we were able to implement this advanced feature into the Runrs app and intend to enhance its functionality in assignment 3.

2) Status tracking and user roles

Private and public clubs have been added to the Runrs app. For a user to join a private club, the owner has to accept your join request. ClubController() has the Join() action method and that creates the membership record when a user joins a club. If a user is joining a private club, then the membership is still created but is read as pending until the owner accepts or rejects them. The owner can see all the pending and that's done with the ManageRequests() method. The pending status is updated by using either with the AprovedRequest() or RejectMethod. There is also a Leave() method which deletes the membership of a user.

4. Constraints & Strategies (Byron & Mo)

Byron

During Assignment (2), I encountered several constraints and implemented a variety of strategies to ensure our web application was of a high standard. The main constraints include:

Constraints

Time Management

Being able to manage my time during this assignment was the biggest constraint I faced. I had quite big expectations for what we could implement into the Runrs application and planned to implement quite complex features during this assignment. However, the further along we got

into the application, the more those ideas didn't seem feasible due to the time I needed to spend completing other assignments to an equally as high standard.

Strategy: Prioritised the core features of our application to provide a working iteration of our application for assignment 2. Core features include the authentication system, CRUD operations for clubs, club member management and working UI components.

Concept Familiarity

Although I am fairly familiar with the C# language due to my experience in previous assignments and personal use, I still found it hard to understand some concepts of MVC and the repository/UoW patterns. Since it was my first time implementing the patterns, I leveraged the work we've been doing on the Bulky web app and referenced a lot of it in our application. If I was to implement these concepts again, without reference to our lab session work, a more detailed understanding will be required.

Strategy: Referenced the work we have been doing for the Bulky web application. The examples that were provided in our lab sessions has greatly assisted us in implementing the repository and UoW patterns.

Password Storage

Our original idea was to implement hashed passwords into our application to promote secure coding across our application. This can be seen with the password property which was named "PasswordHash." However, after getting partway through the development of our application, we decided to backlog the password hashing for the final assignment as we were having issues implementing this security feature. As of the end of assignment 2, the user passwords are being stored as plain text.

Strategy: We have decided to backlog this feature for assignment 3 to ensure the core features of our application are delivered. We have installed the BCrypt package to prepare for full hashing implementation.

Client-Side Validation

As an assignment requirement, jQuery validations on the client-side should have been implemented for the application. We have implemented data annotations into our application on the server-side; however, client-side validations were one feature we overlooked and were not able to implement into the Runrs application as of the end of assignment 2. We are hoping to utilize this feature in the assignment 3 iteration of our application.

Strategy: To alleviate any potential validation issues on the client-side, we have gone through our application and strengthened the server-side validations. This can be seen when observing the data annotations across the model classes, as well as the validation checks on our controller methods "***ModelState.IsValid***."

Mo

Unfamiliarity with coding

One constraint I had in the assignment was trying to remember what every method and code block did. I took a two-year break from school and in that break I lost a lot of the muscle memory I built up over the years.

Strategy: I was able to regain a lot of it by working on both the assignment and the lab work.

Time constraint

The amount of time we had to complete was one of our biggest constraints. Starting out we had a large scope for our app and trying to implement every one of them in only a few weeks while juggling other class assignments was impracticable.

Strategy: To finish the app in time we had to shorten the scope of it. We took out password hashing from the app because we didn't have enough time to fix the bugs and we didn't implement the map feature.

No home PC

since I didn't have a home pc I had to use my old laptop to do the assignment at home. unfortunately, my laptop was prone to slow and would crash every now and then. This made it hard to be constant with my work.

Strategy

I started going to campus on my off days and staying back an extra hour or two after class so I could work on the assignment. This way I would not be behind on the assignment.